

PRACTICAL : 01

Aim:

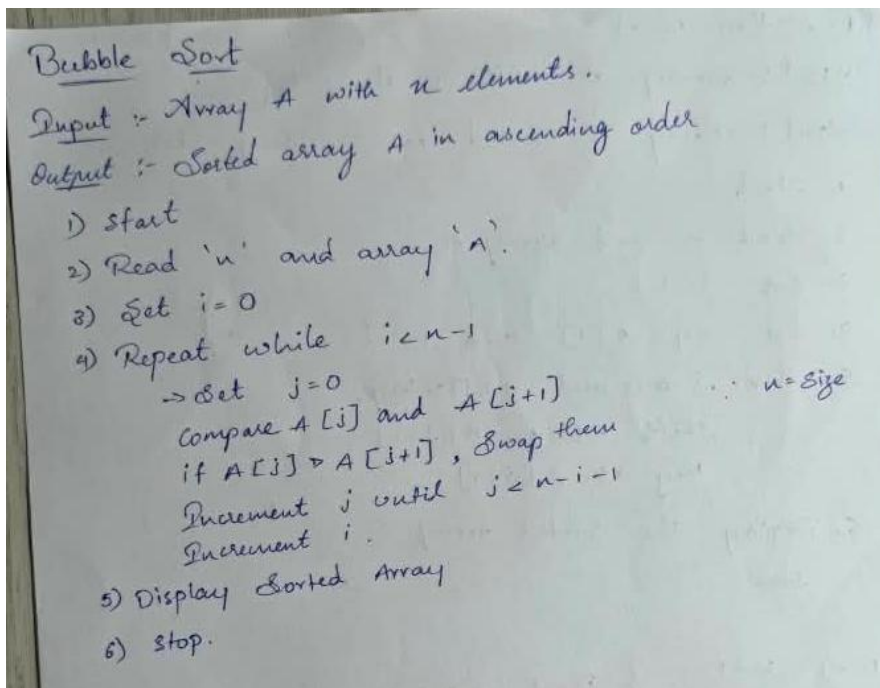
Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1. Bubble Sort

Aim:

To implement Bubble Sort to sort a given array of elements in ascending order.

Algorithm:



Code:

```
#include <iostream>
```

```
using namespace std;

class BubbleSort {
public:
    int size;
    int arr[100];

    void ArrayInput() {
        cout << "Enter the size of the array: ";
        cin >> size;

        cout << "Enter the array elements: ";
        for (int i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

    void printarray() {
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

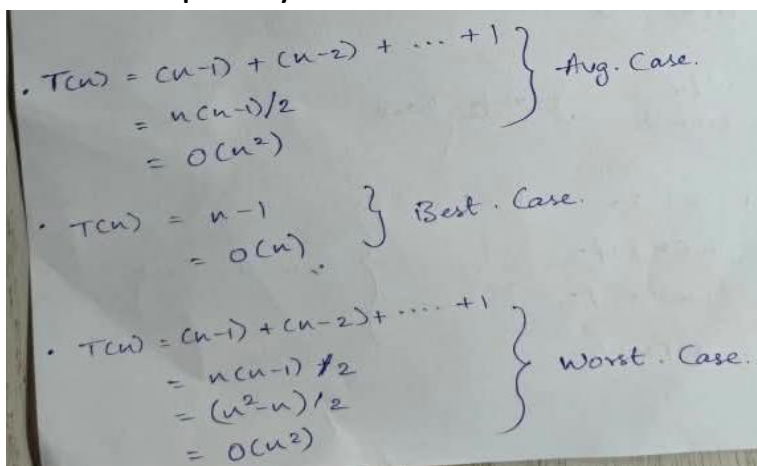
    void bubbleLogic() {
        for (int i = 0; i < size - 1; i++) {
            for (int j = 0; j < size - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
};
```

```
int main() {  
    BubbleSort bs;  
    bs.ArrayInput();  
  
    cout << "Before sorting: ";  
    bs.printarray();  
  
    bs.bubbleLogic();  
  
    cout << "After sorting: ";  
    bs.printarray();  
  
    return 0;  
}
```

Output:

```
Enter the size of the array: 6  
Enter the array elements: 34 56 54 45 23  
4  
Before sorting: 34 56 54 45 23 4  
After sorting: 4 23 34 45 54 56
```

Time Complexity:



• $T(n) = (n-1) + (n-2) + \dots + 1$ } Avg. Case.
 $= n(n-1)/2$
 $= O(n^2)$

• $T(n) = n-1$ } Best Case.
 $= O(n)$

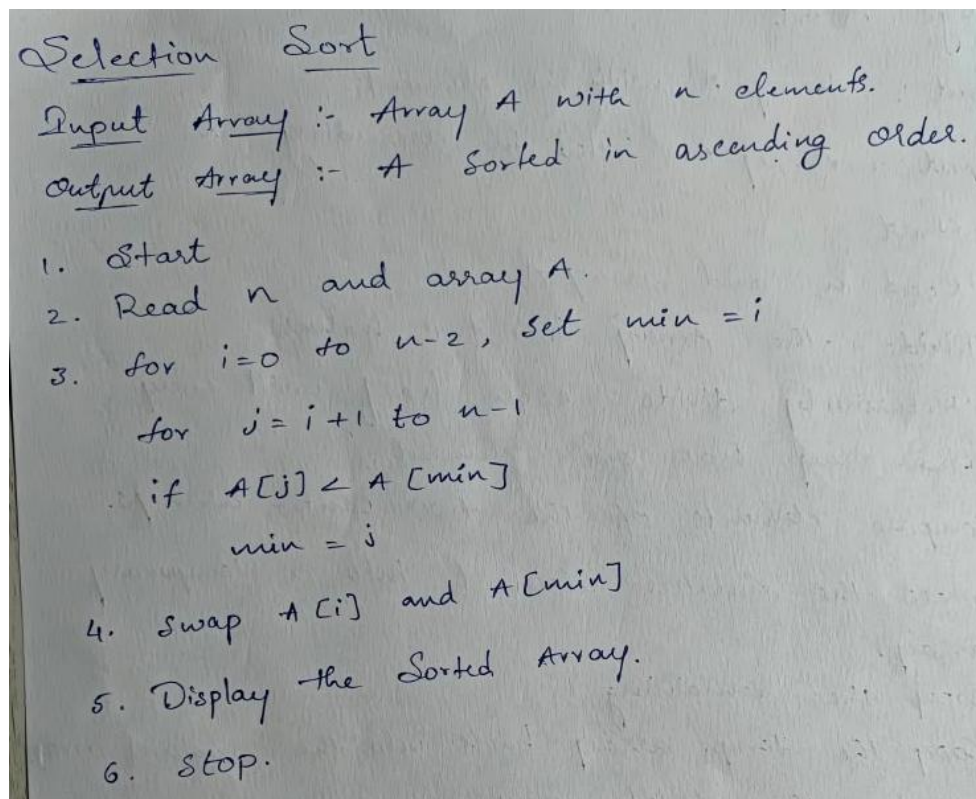
• $T(n) = (n-1) + (n-2) + \dots + 1$ } Worst Case.
 $= n(n-1)/2$
 $= (n^2 - n)/2$
 $= O(n^2)$

2. Selection Sort

Aim:

To implement Selection Sort to sort a given array of elements in ascending order.

Algorithm:



Code:

```
#include <iostream>
using namespace std;

class SelectionSort {
public:
    int size;
    int arr[100];
```

```
void ArrayInput() {
    cout << "Enter the size of the array: ";
    cin >> size;

    cout << "Enter the array elements: ";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

void printarray() {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void selectionLogic() {
    for (int i = 0; i < size - 1; i++) {
        int min = i;

        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[min]) {
                min = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
};

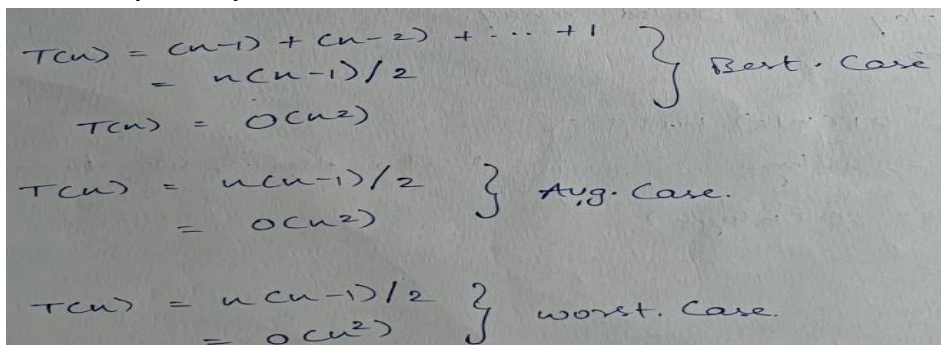
int main() {
    SelectionSort ss;
    ss.ArrayInput();
```

```
cout << "Before sorting: ";  
ss.printarray();  
  
ss.selectionLogic();  
  
cout << "After sorting: ";  
ss.printarray();  
  
return 0;  
}
```

Output:

```
Enter the size of the array: 5  
Enter the array elements: 23 21 12 1 3 19  
Before sorting: 23 21 12 1 3  
After sorting: 1 3 12 21 23
```

Time Complexity:



Handwritten notes showing the time complexity of Selection Sort:

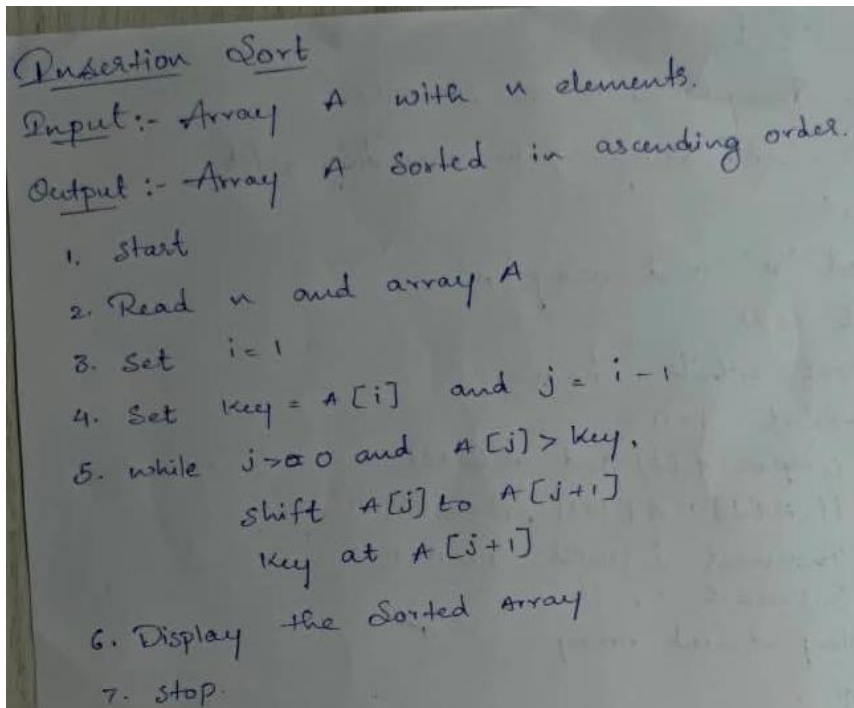
- Best Case: $T(n) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2 = O(n^2)$
- Avg. Case: $T(n) = n(n-1)/2 = O(n^2)$
- Worst Case: $T(n) = n(n-1)/2 = O(n^2)$

3. Insertion Sort

Aim:

To implement Insertion Sort to sort a given array of elements in ascending order.

Algorithm:



Code:

```
#include <iostream>
using namespace std;
```

```
class InsertionSort {
public:
    int size;
    int arr[100];

    void ArrayInput() {
```

```
cout << "Enter the size of the array: ";
cin >> size;

cout << "Enter the array elements: ";
for (int i = 0; i < size; i++) {
    cin >> arr[i];
}

void printarray() {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void insertionLogic() {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
};

int main() {
    InsertionSort is;
    is.ArrayInput();

    cout << "Before sorting: ";
    is.printarray();
}
```

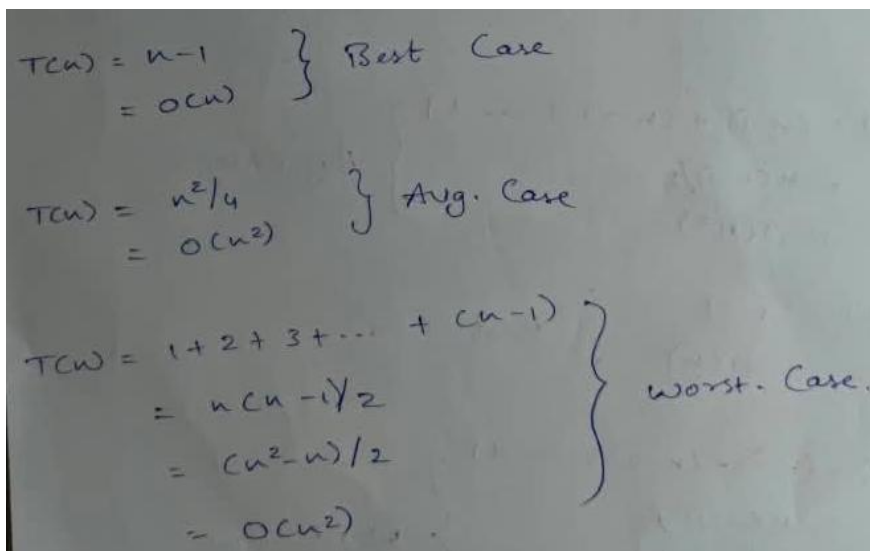


```
is.insertionLogic();  
  
cout << "After sorting: ";  
is.printarray();  
  
return 0;  
}
```

Output:

```
Enter the size of the array: 4  
Enter the array elements: 10 19 1 8  
Before sorting: 10 19 1 8  
After sorting: 1 8 10 19
```

Time Complexity:



Handwritten time complexity analysis for Insertion Sort:

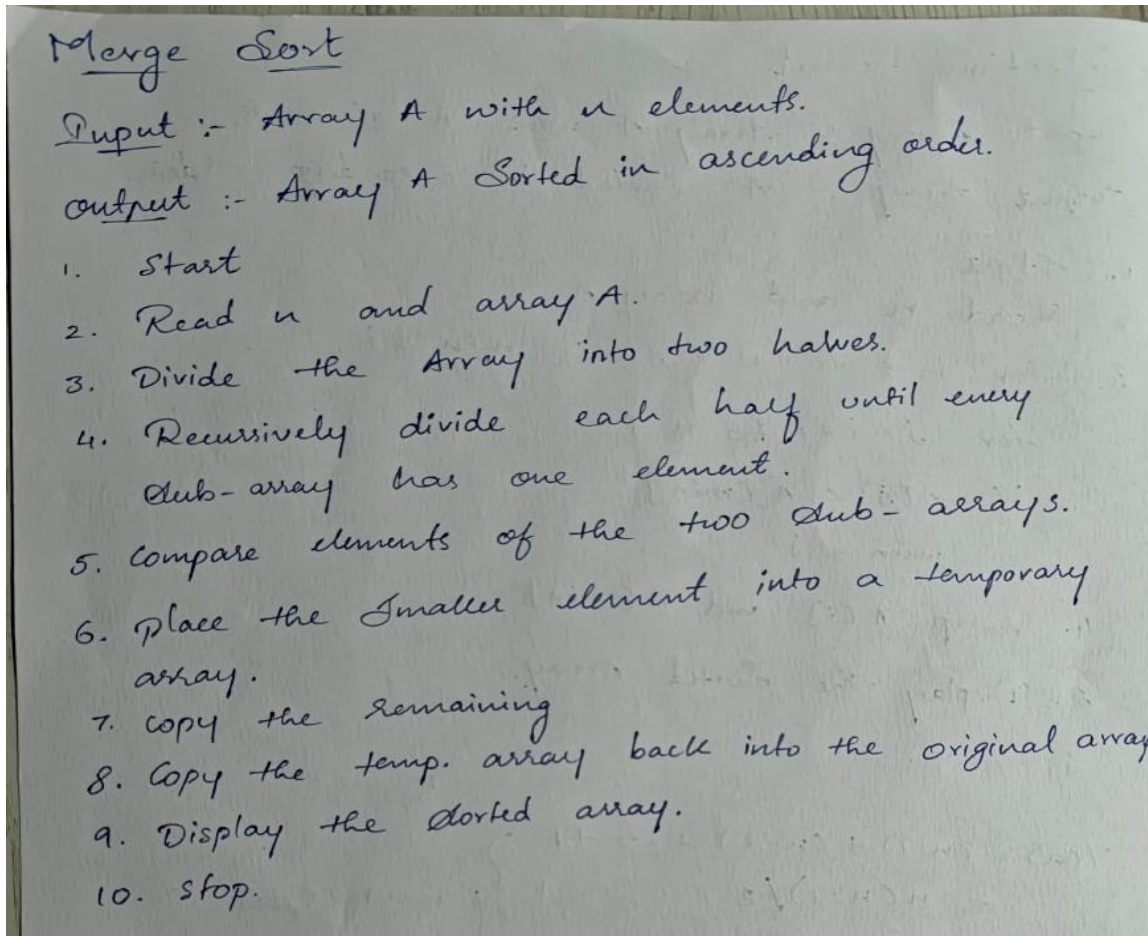
- Best Case: $T(n) = n - 1$, $= O(n)$
- Avg. Case: $T(n) = \frac{n^2}{4}$, $= O(n^2)$
- Worst Case: $T(n) = 1 + 2 + 3 + \dots + (n - 1)$
 $= \frac{n(n - 1)}{2}$
 $= \frac{(n^2 - n)}{2}$
 $= O(n^2)$

4. Merge Sort

Aim:

To implement Merge Sort to sort a given array of elements in ascending order.

Algorithm:



Code:

```
#include <iostream>
using namespace std;

class MergeSort {
public:
    int size;
    int arr[100];

    void ArrayInput() {
        cout << "Enter the size of the array: ";
```

```
cin >> size;

cout << "Enter the array elements: ";
for (int i = 0; i < size; i++) {
    cin >> arr[i];
}
}

void printarray() {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void merge(int low, int mid, int high) {
    int temp[100];
    int i = low;
    int j = mid + 1;
    int k = low;

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    while (j <= high) {
        temp[k++] = arr[j++];
    }
}
```

```
        for (i = low; i <= high; i++) {
            arr[i] = temp[i];
        }
    }

void mergeLogic(int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;

        mergeLogic(low, mid);
        mergeLogic(mid + 1, high);
        merge(low, mid, high);
    }
}

};

int main() {
    MergeSort ms;
    ms.ArrayInput();

    cout << "Before sorting: ";
    ms.printarray();

    ms.mergeLogic(0, ms.size - 1);

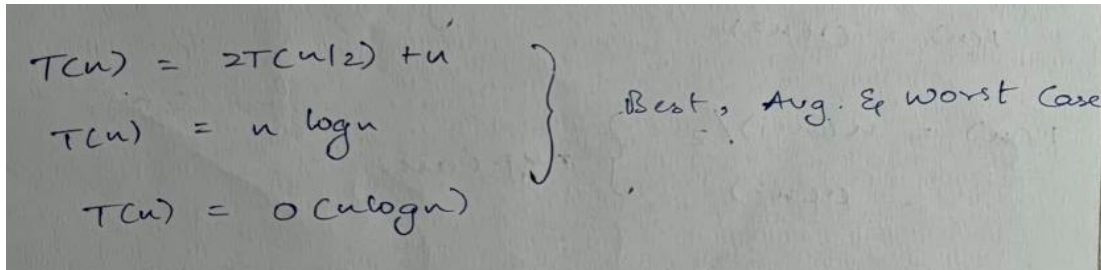
    cout << "After sorting: ";
    ms.printarray();

    return 0;
}
```

Output:

```
Enter the size of the array: 7
Enter the array elements: 9 70 33 90 46 6 59
Before sorting: 9 70 33 90 46 6 59
After sorting: 6 9 33 46 59 70 90
```

Time Complexity:



Handwritten notes showing the recurrence relation and time complexity for Quick Sort:

$$T(n) = 2T(n/2) + n$$
$$T(n) = n \log n$$
$$T(n) = O(n \log n)$$

Best, Avg. & Worst Case

5. Quick Sort

Aim:

To implement Quick Sort to sort a given array of elements in ascending order.

Algorithm:

Quick Sort

Input :- Array 'A' with 'n' elements.

Output :- Sorted Array A in ascending order.

1. Start
2. Read n and array A.
3. Call Quick Sort (A, low = 0, high = $n-1$).
4. Select the last element A[high] as the pivot.
5. Set $i = \text{low} - 1$
6. For $j = \text{low}$ to $\text{high} - 1$:
 - if $A[j] \leq \text{pivot}$;
 - Increment i
 - Swap $A[i]$ & $A[j]$
7. Swap $A[i+1]$ and the pivot element A[high]
8. Place the pivot in its correct position.
9. Apply Quick Sort recursively to:
 - left part: low to pivot position - 1
 - Right part: pivot position + 1 to high.
10. Repeat steps 4 to 9 until $\text{low} \geq \text{high}$.
11. Display Sorted Array 'A'.
12. Stop.

Code:

```
#include <iostream>
using namespace std;

class QuickSort {
public:
    int size;
    int arr[100];

    void ArrayInput() {
        cout << "Enter the size of the array: ";
        cin >> size;

        cout << "Enter the array elements: ";
        for (int i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

    void printarray() {
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    int partition(int low, int high) {
        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
```

```
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickLogic(int low, int high) {
    if (low < high) {
        int loc = partition(low, high);

        quickLogic(low, loc - 1);
        quickLogic(loc + 1, high);
    }
}

};

int main() {
    QuickSort qs;
    qs.ArrayInput();

    cout << "Before sorting: ";
    qs.printarray();

    qs.quickLogic(0, qs.size - 1);

    cout << "After sorting: ";
    qs.printarray();

    return 0;
}
```

Output:


```
Enter the size of the array: 5
Enter the array elements: 99 78 34 2 1
Before sorting: 99 78 34 2 1
After sorting: 1 2 34 78 99
```

Time Complexity:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= O(n \log n) \end{aligned} \quad \left. \vphantom{\begin{aligned} T(n) &= 2T(n/2) + n \\ &= O(n \log n) \end{aligned}} \right\} \text{Best. Case}$$
$$T(n) = O(n \log n) \rightarrow \text{Avg. Case}$$
$$\begin{aligned} T(n) &= T(n-1) + n \\ &= n + (n-1) + (n-2) + \dots + 1 \\ &= n(n+1)/2 \end{aligned} \quad \left. \vphantom{\begin{aligned} T(n) &= T(n-1) + n \\ &= n + (n-1) + (n-2) + \dots + 1 \\ &= n(n+1)/2 \end{aligned}} \right\} \text{Worst. Case}$$
$$T(n) = O(n^2)$$